

Grundlagen der Programmierung:

7. Programmieren in C

Tutoren:

Daniel Grillmeyer, Ludwig Friedl, Jan Kerber, Jennifer Muck

1. Lösung Programmieraufgabe 2
2. Puzzler
3. Unterschiede zwischen Java und C
4. Speicherverwaltung
5. Strukturen und Speicherzugriff
6. Größe von Datentypen im Speicher
7. Beispielprogramm

LÖSUNG PROGRAMMIERAUFGABE 2

Lösung Programmieraufgabe 2

- **Nicht Teil des Skripts – wird nur in der Übung besprochen**

PUZZLER

Puzzler

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int array[30];
6     int i;
7     int sum;
8     sum = 0;
9     for (i = 0; i < 30; i++)
10         sum = sum + array[i];
11     printf("Inhalt: %d", sum);
12     return 0;
13 }
```

Was wird ausgegeben?

30

Übersetzerfehler
(Compilererror)

Laufzeitfehler
(Exception)

Nicht vorhersagbar

Puzzler – Auflösung

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int array[30];
6     int i;
7     int sum;
8     sum = 0;
9     for (i = 0; i < 30; i++)
10         sum = sum + array[i];
11     printf("Inhalt: %d", sum);
12     return 0;
13 }
```

Was wird ausgegeben?

30

Übersetzerfehler
(Compilererror)

Laufzeitfehler
(Exception)

Nicht vorhersagbar

Puzzler – Auflösung

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int array[30];
6      int i;
7      int sum;
8      sum = 0;
9      for (i = 0; i < 30; i++)
10         sum = sum + array[i];
11     printf("Inhalt: %d", sum);
12     return 0;
13 }
```

Was wird ausgegeben?

30

Übersetzerfehler
(Compilererror)

Laufzeitfehler
(Exception)

Nicht vorhersagbar

➤ Warum?

- C-Variablen werden nicht automatisch auf 0 initialisiert.
- Es können alte Daten im Speicher stehen.

UNTERSCHIEDE ZWISCHEN JAVA UND C

Unterschiede zwischen Java und C

➤ Syntaktische Unterschiede in den Datentypen:

```
1 public static void main(String[] args) {
2     String text = "Let's Code";
3     char c = ':';
4     text = text + c;
5     System.out.print(text);
6     for (int i = 0; i < 10; i++) {
7         System.out.printf(" %d", i);
8     }
9     if (false) {
10        System.out.print("false");
11    }
12    int a = 5;
13    function(a);
14 }
```

```
1 int main() {
2     char text[32] = "Let's Code";
3     char c[] = ":";
4     strcat(text, c);
5     printf(text);
6     for (int i = 0; i < 10; i++) {
7         printf(" %d", i);
8     }
9     if (0) {
10        printf("false");
11    }
12    int a = 5;
13    function(&a);
14 }
```

Unterschiede zwischen Java und C

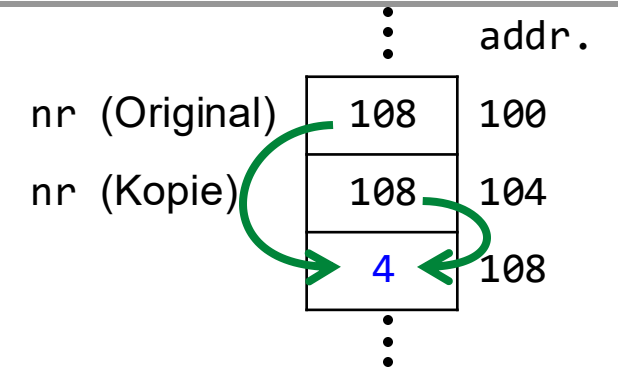
Eingabe: Kopie eines Pointers
Setze Inhalt des Pointers auf 4

```
15 void function(int* nr) {  
16     *nr = 4;  
17 }
```

Unterschiede zwischen Java und C

Eingabe: Kopie eines Pointers
Setze Inhalt des Pointers auf 4

```
15 void function(int* nr) {  
16     *nr = 4;  
17 }
```



Unterschiede zwischen Java und C

Eingabe: Kopie eines Pointers
Setze Inhalt des Pointers auf 4

```
15 void function(int* nr) {  
16     *nr = 4;  
17 }
```

Eingabe: Kopie eines Pointers
Lass diese Kopie auf Adresse 4 zeigen

```
15 void function(int* nr) {  
16     nr = 4;  
17 }
```

Unterschiede zwischen Java und C

Eingabe: Kopie eines Pointers
Setze Inhalt des Pointers auf 4

```
15 void function(int* nr) {  
16     *nr = 4;  
17 }
```

Eingabe: Kopie eines Pointers
Lass diese Kopie auf Adresse 4 zeigen

```
15 void function(int* nr) {  
16     nr = 4;  
17 }
```

		addr.
		4
nr (Original)	108	100
nr (Kopie)	4	104

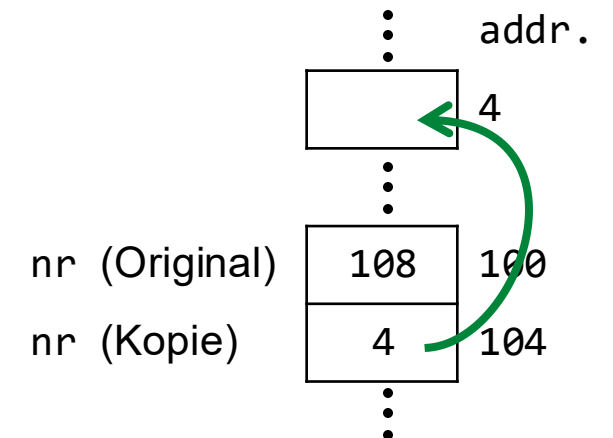
Unterschiede zwischen Java und C

Eingabe: Kopie eines Pointers
Setze Inhalt des Pointers auf 4

```
15 void function(int* nr) {  
16     *nr = 4;  
17 }
```

Eingabe: Kopie eines Pointers
Lass diese Kopie auf Adresse 4 zeigen

```
15 void function(int* nr) {  
16     nr = 4;  
17 }
```



Unterschiede zwischen Java und C

Eingabe: Kopie eines Pointers
Setze Inhalt des Pointers auf 4

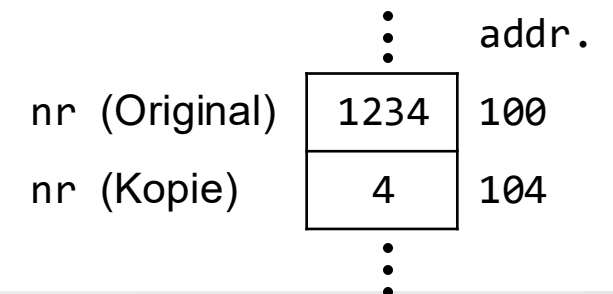
```
15 void function(int* nr) {  
16     *nr = 4;  
17 }
```

Eingabe: Kopie eines Pointers
Lass diese Kopie auf Adresse 4 zeigen

```
15 void function(int* nr) {  
16     nr = 4;  
17 }
```

Eingabe: Kopie einer Zahl
Lass diese Kopie den Wert 4 annehmen

```
15 void function(int nr) {  
16     nr = 4;  
17 }
```



SPEICHERVERWALTUNG

Zentrale Begriffe

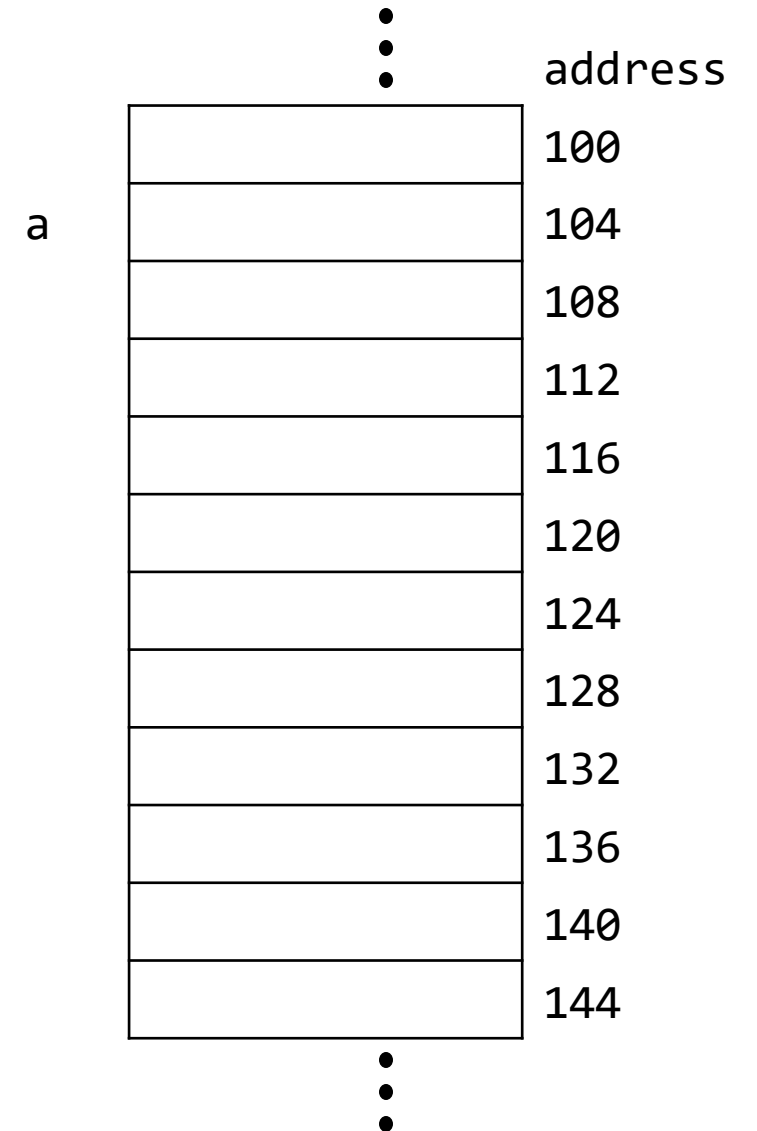
- Pointer: Variable, die eine *Adresse* einer anderen Variable enthält
- Variable: Speicherzelle(n), die einen Wert enthalten
- Referenzierung: Adresse einer Variable erhalten
- Dereferenzierung: Zielwert eines Pointers erhalten

Speicherverwaltung

Beispiele:

- Erstelle eine Variable a mit Wert 5

```
int a = 5;
```

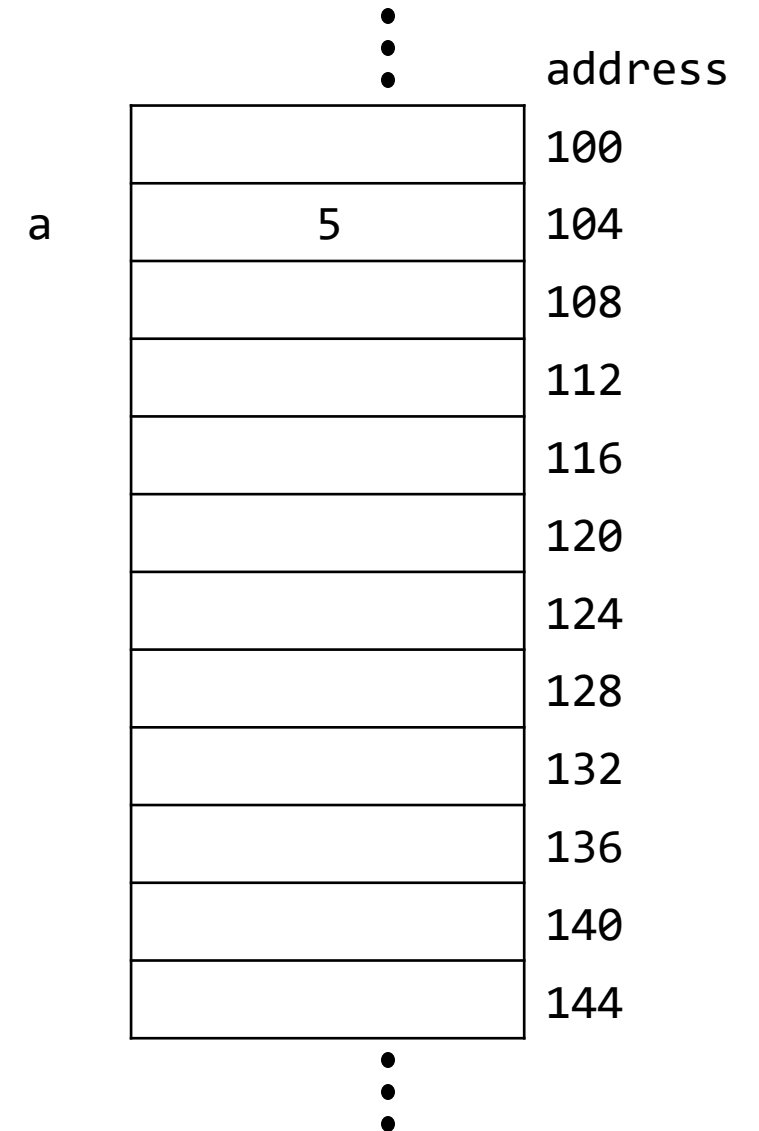


Speicherverwaltung

Beispiele:

- Erstelle eine Variable a mit Wert 5

```
int a = 5;
```



Speicherverwaltung

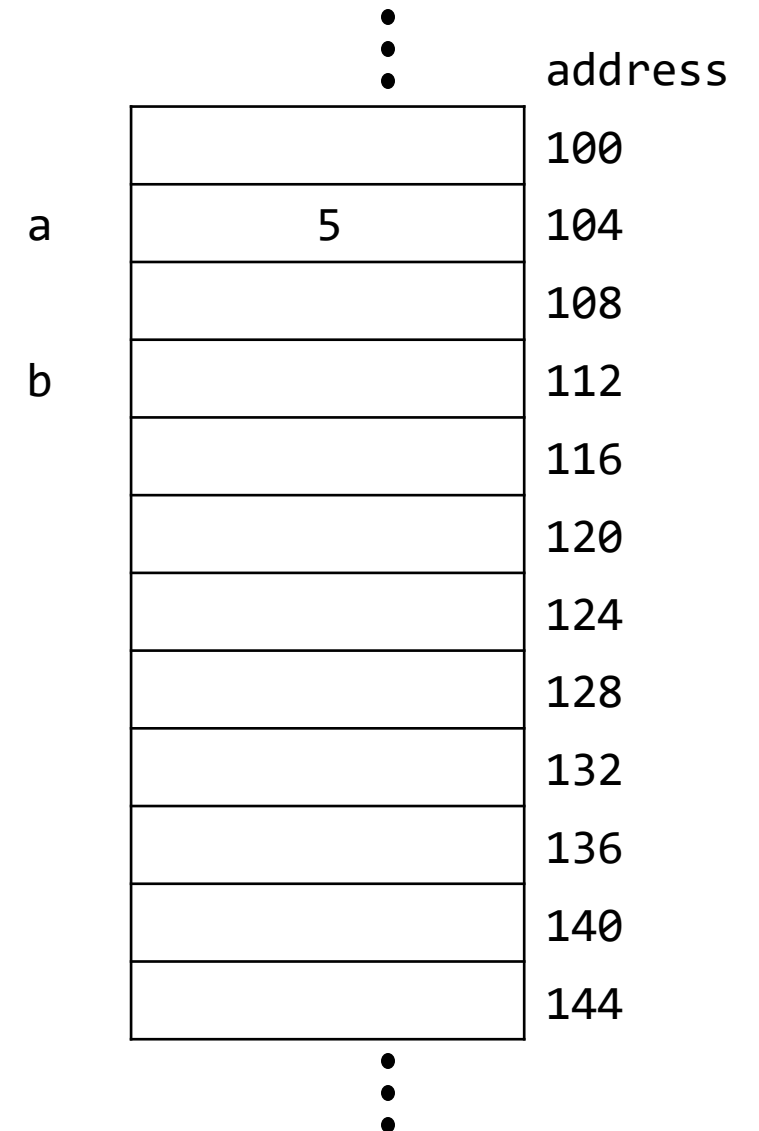
Beispiele:

➤ Erstelle eine Variable a mit Wert 5

```
int a = 5;
```

➤ Erstelle einen Pointer b auf eine Zahl

```
int* b;
```



Speicherverwaltung

Beispiele:

➤ Erstelle eine Variable a mit Wert 5

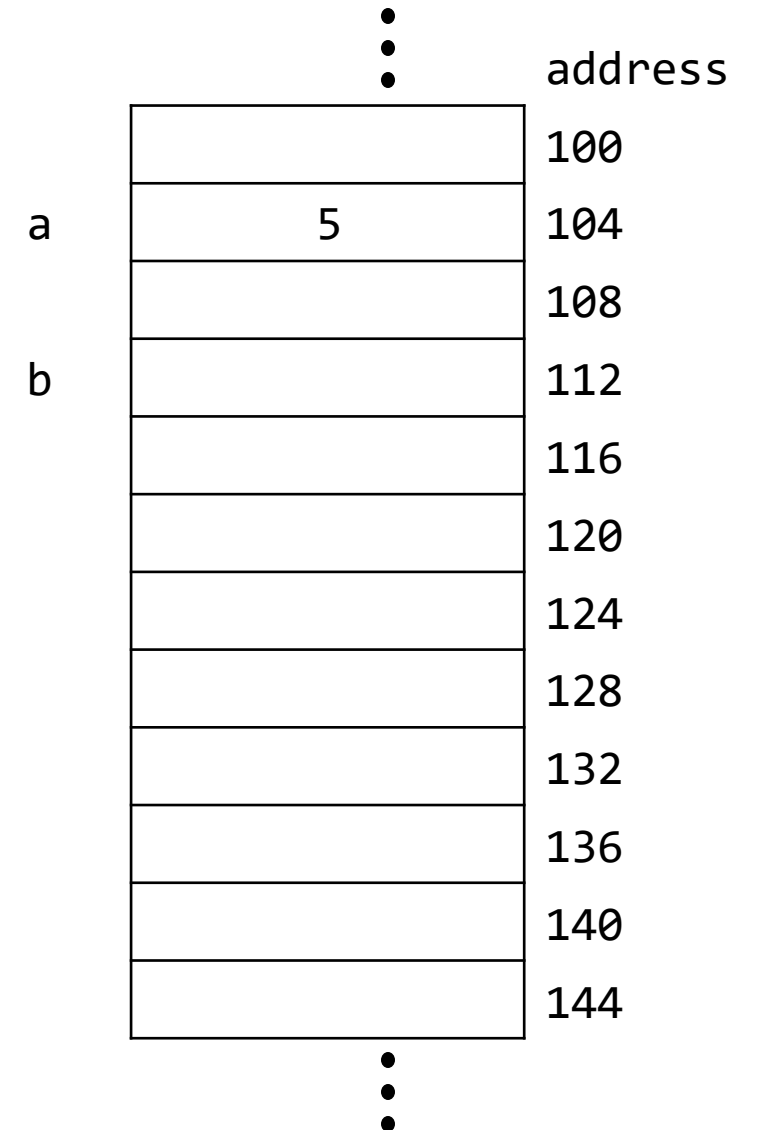
```
int a = 5;
```

➤ Erstelle einen Pointer b auf eine Zahl

```
int* b;
```

➤ Die Adresse von a

```
&a;
```



Speicherverwaltung

Beispiele:

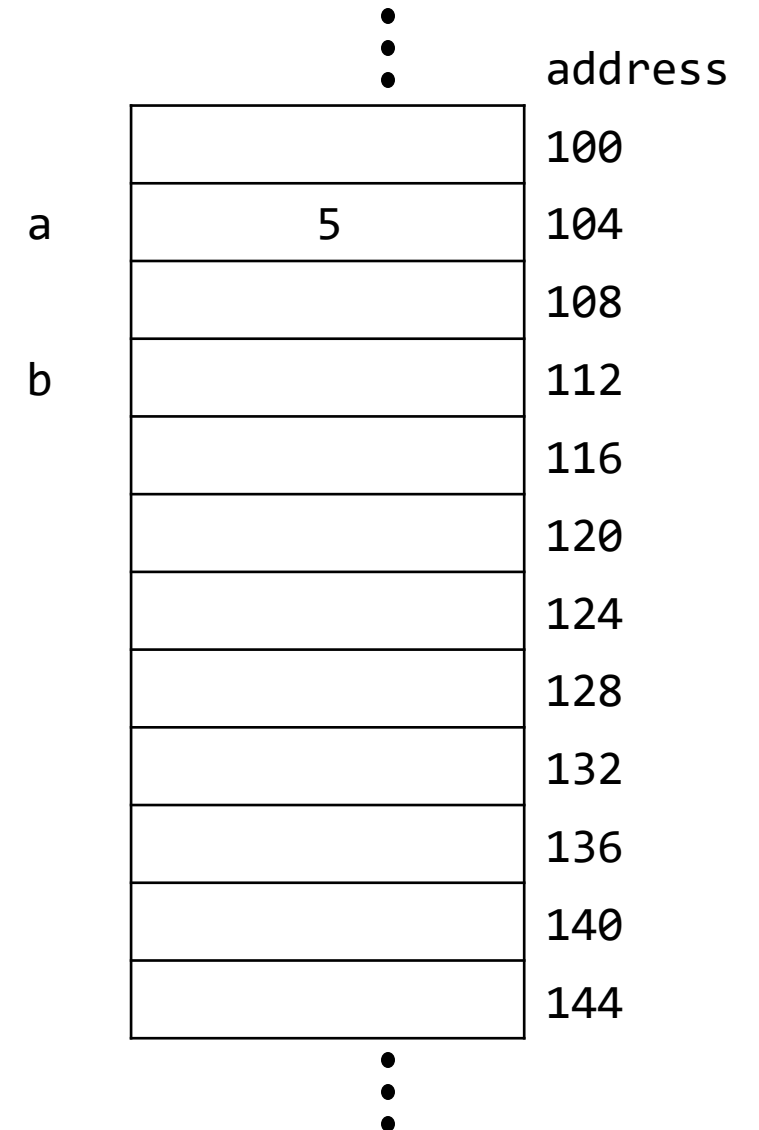
- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a

```
int a = 5;
```

```
int* b;
```

```
&a;
```

== 104



Speicherverwaltung

Beispiele:

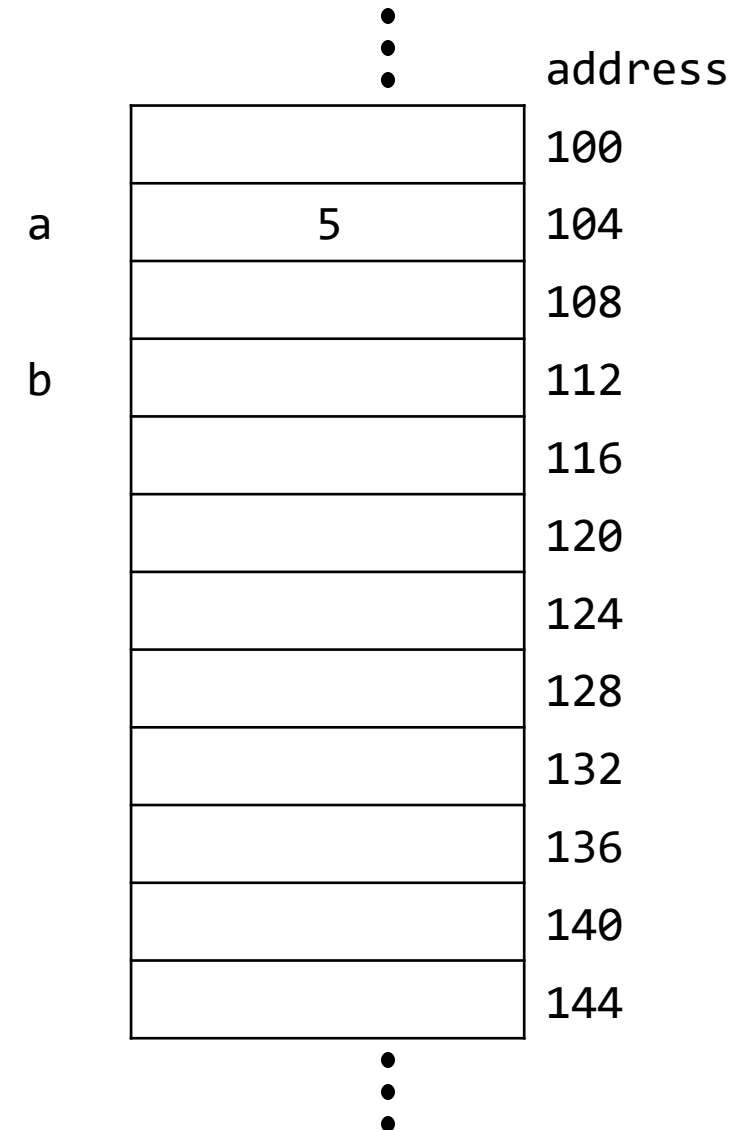
- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a == 104
- Pointer b zeigt auf die Zahl a

```
int a = 5;
```

```
int* b;
```

```
&a;
```

```
b = &a;
```



Speicherverwaltung

Beispiele:

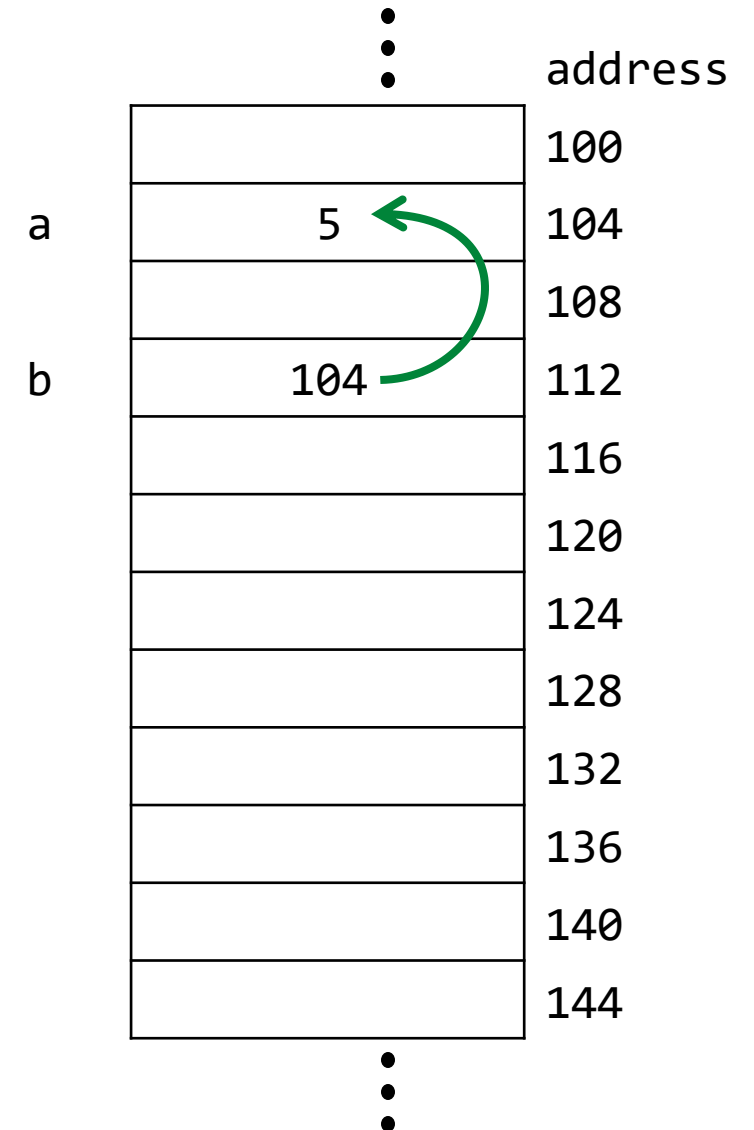
- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a == 104
- Pointer b zeigt auf die Zahl a

```
int a = 5;
```

```
int* b;
```

```
&a;
```

```
b = &a;
```



Speicherverwaltung

Beispiele:

- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a == 104
- Pointer b zeigt auf die Zahl a
- Die Zahl, auf die b zeigt

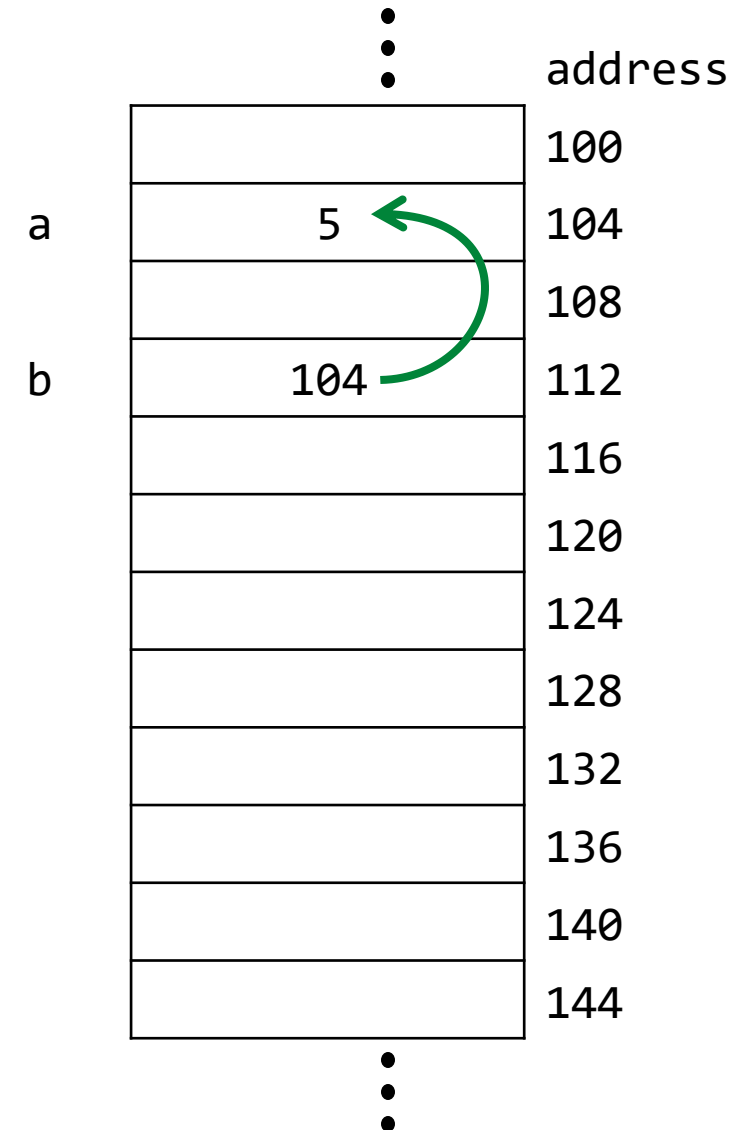
```
int a = 5;
```

```
int* b;
```

```
&a;
```

```
b = &a;
```

```
*b;
```



Speicherverwaltung

Beispiele:

➤ Erstelle eine Variable a mit Wert 5

```
int a = 5;
```

➤ Erstelle einen Pointer b auf eine Zahl

```
int* b;
```

➤ Die Adresse von a

== 104

```
&a;
```

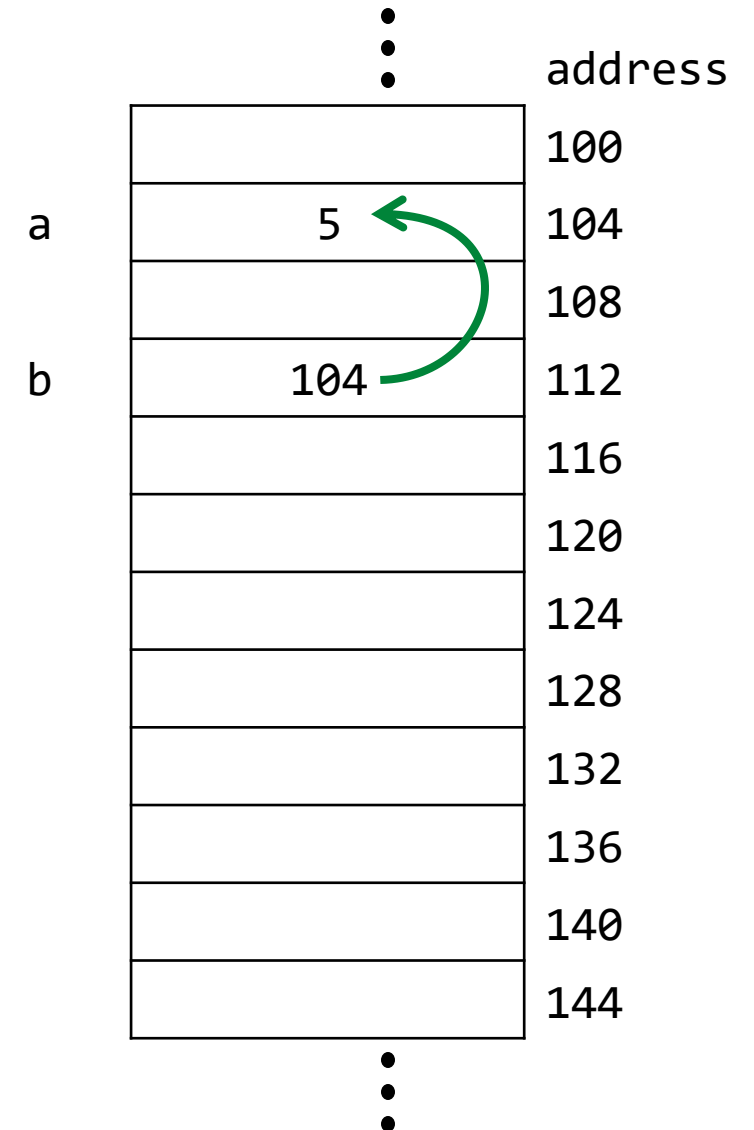
➤ Pointer b zeigt auf die Zahl a

```
b = &a;
```

➤ Die Zahl, auf die b zeigt

== a == 5

```
*b;
```



Speicherverwaltung

Beispiele:

- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a == 104
- Pointer b zeigt auf die Zahl a
- Die Zahl, auf die b zeigt == a == 5
- Lasse einen Pointer c auf die Adresse des Inhaltes von b zeigen

```
int a = 5;
```

```
int* b;
```

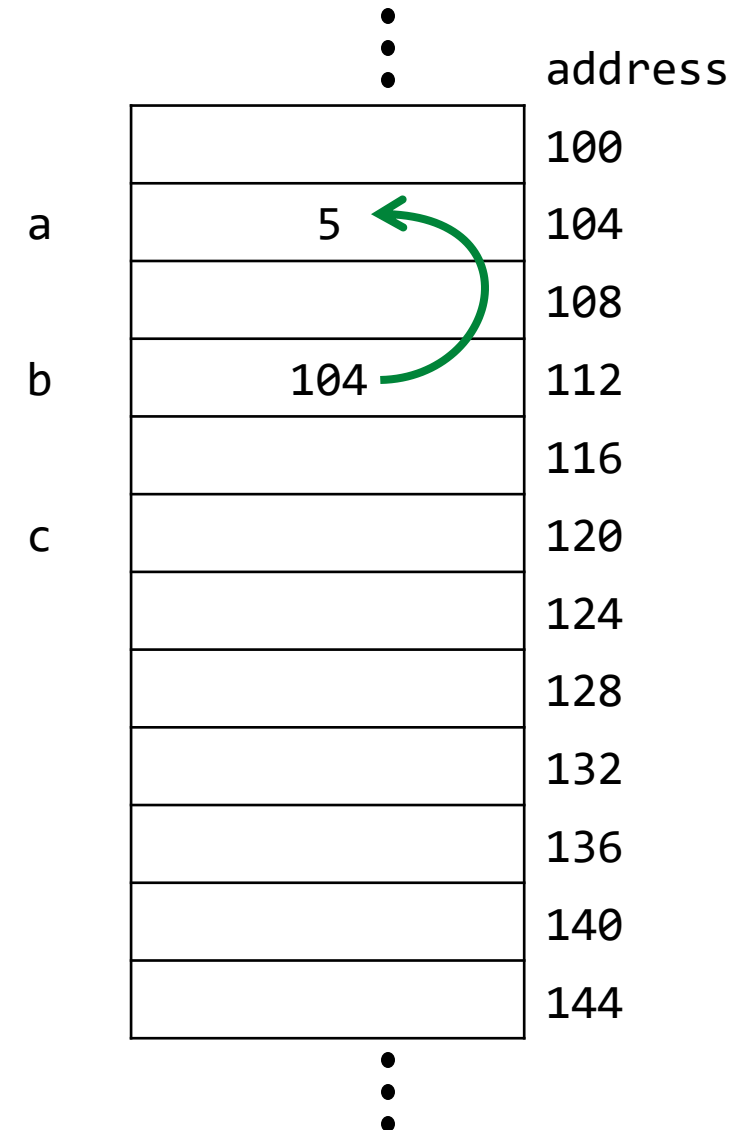
```
&a;
```

```
b = &a;
```

```
*b;
```

```
int* c = &>(*b);
```

```
int* c = b;
```



Speicherverwaltung

Beispiele:

- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a **== 104**
- Pointer b zeigt auf die Zahl a
- Die Zahl, auf die b zeigt **== a == 5**
- Lasse einen Pointer c auf die Adresse des Inhaltes von b zeigen

```
int a = 5;
```

```
int* b;
```

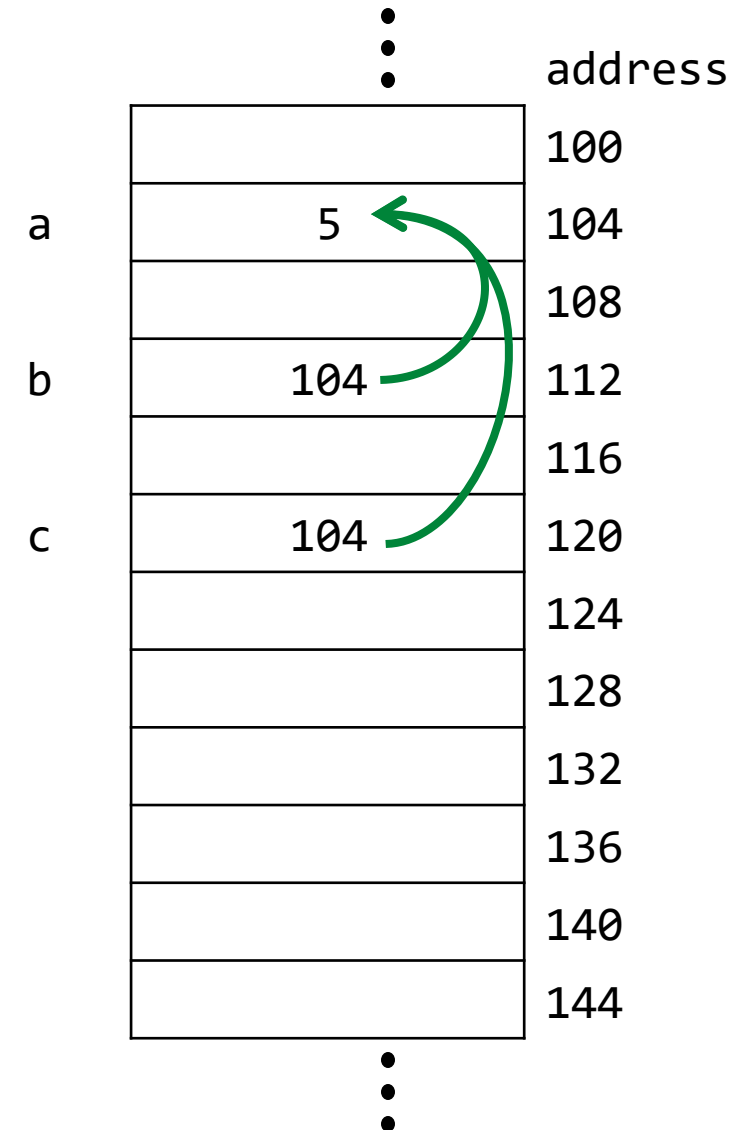
```
&a;
```

```
b = &a;
```

```
*b;
```

```
int* c = &>(*b);
```

```
int* c = b;
```



Speicherverwaltung

Beispiele:

- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a **== 104**
- Pointer b zeigt auf die Zahl a
- Die Zahl, auf die b zeigt **== a == 5**
- Lasse einen Pointer c auf die Adresse des Inhaltes von b zeigen
- Lass einen Pointer-Pointer d auf b zeigen

```
int a = 5;
```

```
int* b;
```

```
&a;
```

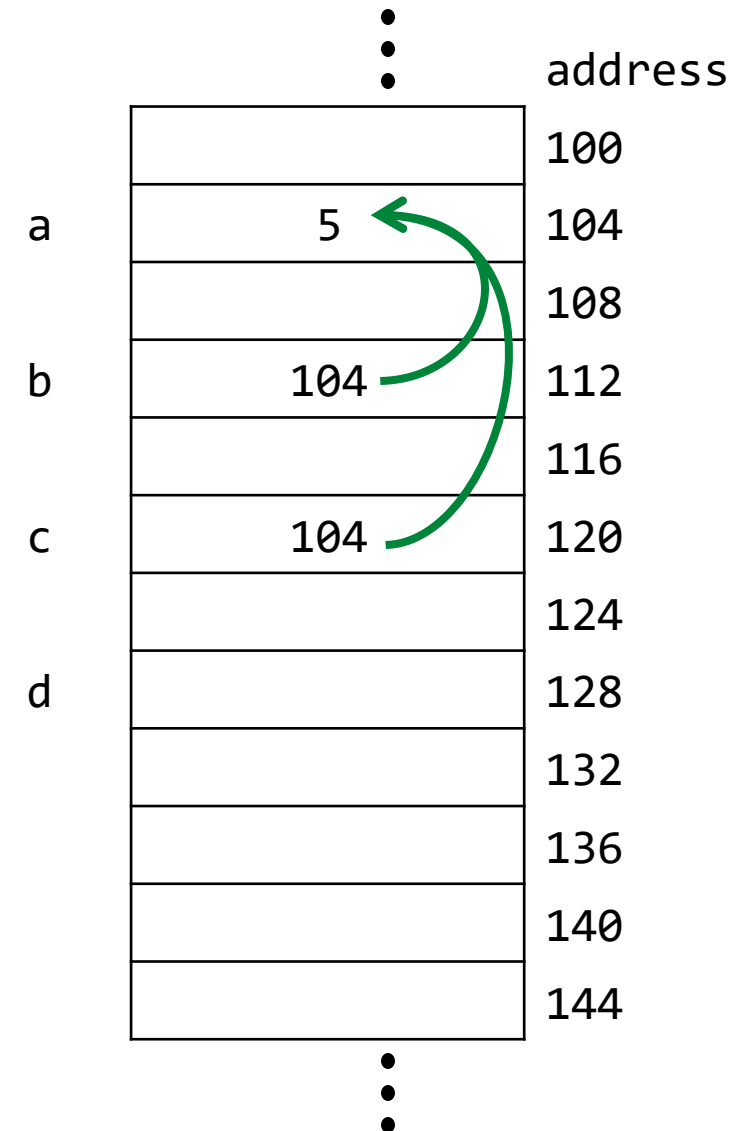
```
b = &a;
```

```
*b;
```

```
int* c = &>(*b);
```

```
int* c = b;
```

```
int** d = &b;
```



Speicherverwaltung

Beispiele:

- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a **== 104**
- Pointer b zeigt auf die Zahl a
- Die Zahl, auf die b zeigt **== a == 5**
- Lasse einen Pointer c auf die Adresse des Inhaltes von b zeigen
- Lass einen Pointer-Pointer d auf b zeigen

```
int a = 5;
```

```
int* b;
```

```
&a;
```

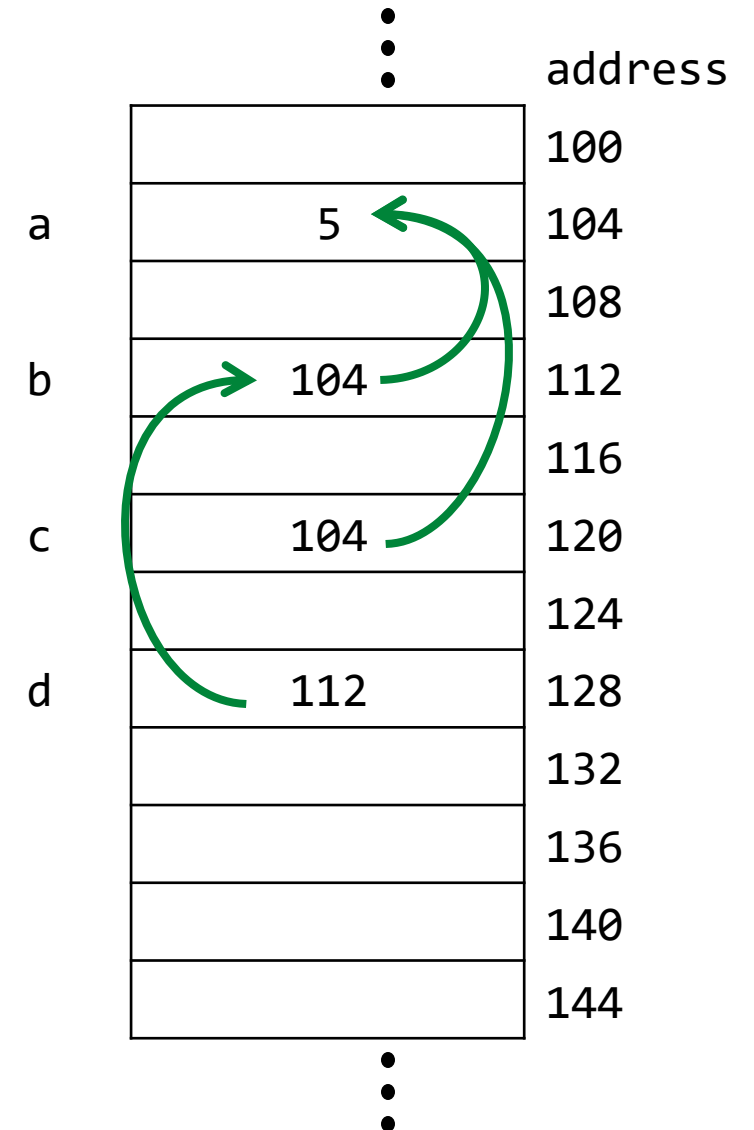
```
b = &a;
```

```
*b;
```

```
int* c = &>(*b);
```

```
int* c = b;
```

```
int** d = &b;
```



Speicherverwaltung

Beispiele:

- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a **== 104**
- Pointer b zeigt auf die Zahl a
- Die Zahl, auf die b zeigt **== a == 5**
- Lasse einen Pointer c auf die Adresse des Inhaltes von b zeigen
- Lass einen Pointer-Pointer d auf b zeigen
- Lass einen Pointer e auf den Inhalt von d zeigen

```
int a = 5;
```

```
int* b;
```

```
&a;
```

```
b = &a;
```

```
*b;
```

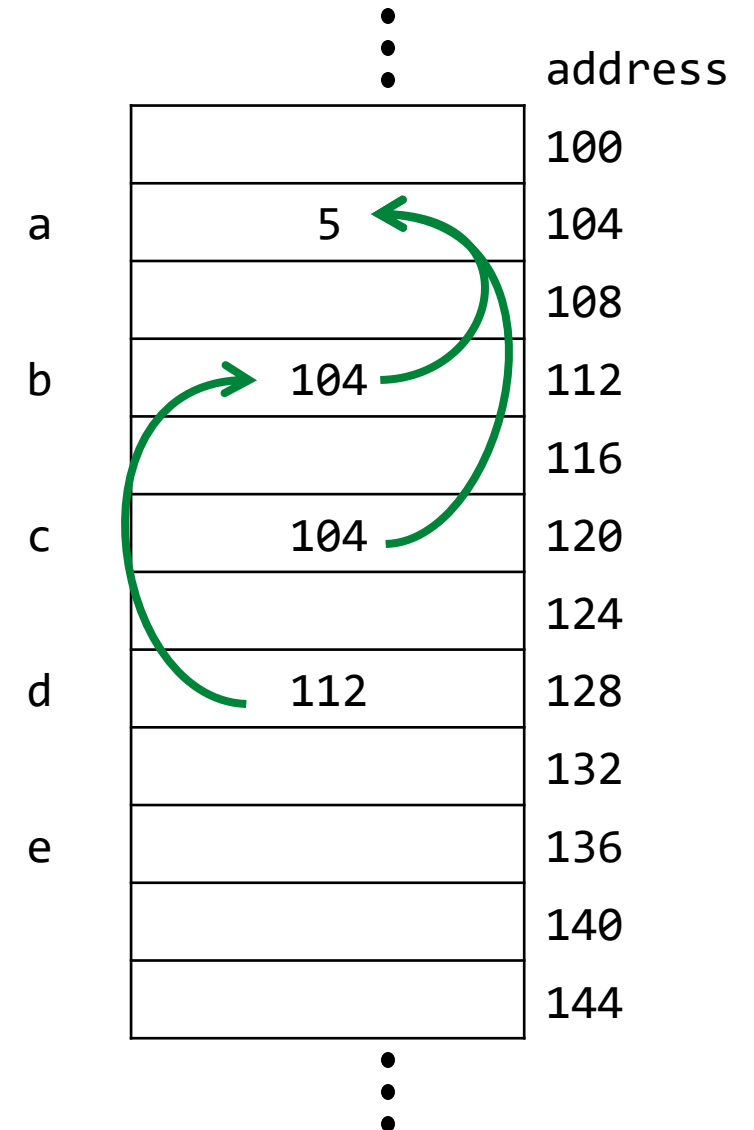
```
int* c = &>(*b);
```

```
int* c = b;
```

```
int** d = &b;
```

```
int* e = *d;
```

```
int* e = &a;
```



Speicherverwaltung

Beispiele:

- Erstelle eine Variable a mit Wert 5
- Erstelle einen Pointer b auf eine Zahl
- Die Adresse von a **== 104**
- Pointer b zeigt auf die Zahl a
- Die Zahl, auf die b zeigt **== a == 5**
- Lasse einen Pointer c auf die Adresse des Inhaltes von b zeigen
- Lass einen Pointer-Pointer d auf b zeigen
- Lass einen Pointer e auf den Inhalt von d zeigen

```
int a = 5;
```

```
int* b;
```

```
&a;
```

```
b = &a;
```

```
*b;
```

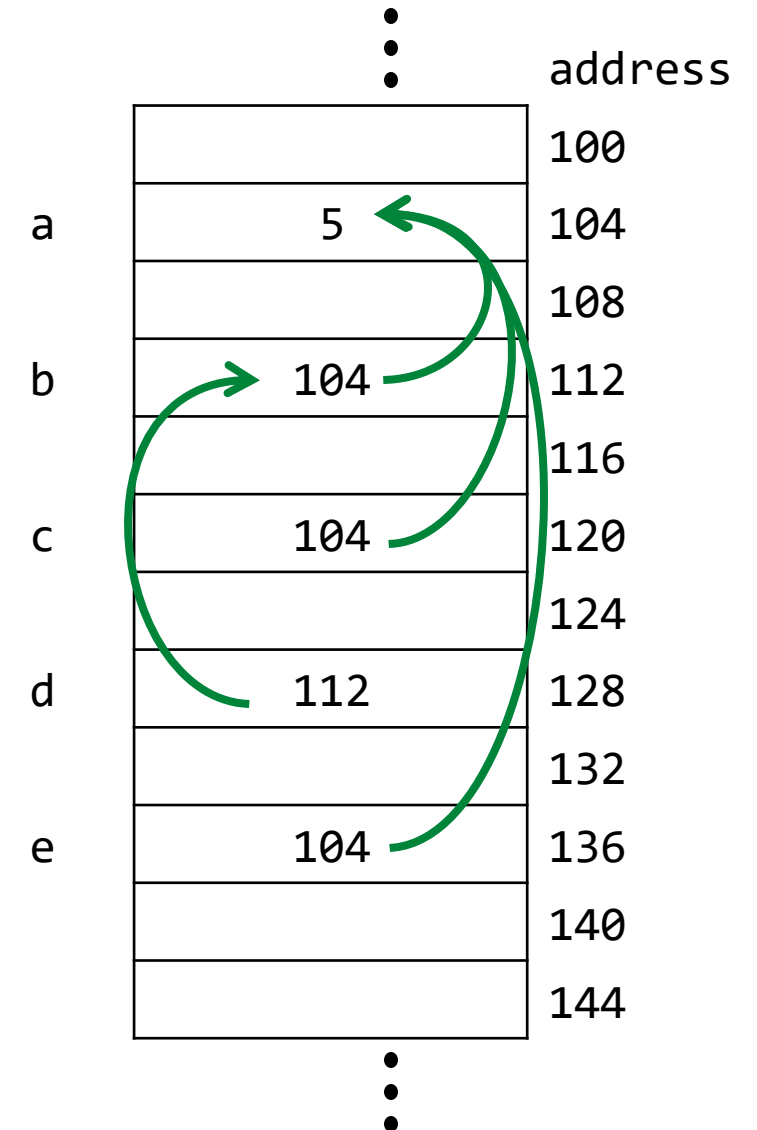
```
int* c = &>(*b);
```

```
int* c = b;
```

```
int** d = &b;
```

```
int* e = *d;
```

```
int* e = &a;
```



STRUKTUREN UND SPEICHERZUGRIFF

Strukturen und Speicherzugriff

- Strukturen (structs) fassen Daten zusammen:

```
1 struct student {  
2     char name[32];  
3     int age;  
4 };  
5  
6 struct student stud1 = {"Max Mustermann", 27};  
7 printf("stud1 name: %s", stud1.name);  
8 printf("stud1 age: %d", stud1.age);
```

- In welchem Speicherbereich wird die Variable stud1 gespeichert?
 - auf dem Stack
- Wie können wir Speicher auf dem Heap allokalieren/freigeben?
 - mit malloc und free (s. nächste Folie)

Strukturen und Speicherzugriff

- Dauerhafte Speicherbelegung auf dem Heap mit malloc:

```
1 struct student* createExampleStudent() {  
2     struct student* pStud = malloc(sizeof(struct student));  
3     strcpy((*pStud).name, "Max Mustermann");  
4     pStud->age = 27;  
5     return pStud;  
6 }
```

- strcpy(char* dest, char* source) kopiert den Inhalt vom String source zu dest
- pStud->age ist äquivalent zu (*pStud).age

- Freigeben des Speichers mit free:

```
7 struct student* derMax = createExampleStudent();  
8 free(derMax);
```

Größe von Datentypen im Speicher

- C kennt unter anderem folgende Datentypen, die wir auch schon aus Java kennen: **int**, **float** und **double**
- Im Gegensatz zu Java ist aber die Größe der Datentypen im Speicher und damit auch deren Wertebereich nicht festgelegt



Arduino Uno



Ein Laptop

Datentyp	Größe in byte (Java)	Größe in byte (C)
int	4	2
float	4	4
double	8	4

Datentyp	Größe in byte (Java)	Größe in byte (C)
int	4	4
float	4	4
double	8	8

Kleine Microcontroller supporten keine Double Precision!

Größe von Datentypen im Speicher – Lessons Learned

- Wenn wir Speicher für Variablen reservieren wollen, ist es **notwendig** den **sizeof-Operator** zu verwenden
- Damit gewährleisten wir, dass unser Programm auf (hoffentlich) jeder Rechenmaschine verwendet werden kann
- Beispiel:
 - DON'T: `int* a = malloc(4)`
 - DO: `int* a = malloc(sizeof(int))`

Pointer-Arithmetik

- Berechnen Sie die Werte der Variablen nach Ausführung jeder Zeile für unsere beiden Testplattformen!



```
1 int* a = 240;  
2 a = a + 2;  
3 int* b = a++;  
4 int* c = --a;  
5 int* d = &a[4];  
6 int* e = (a+d) / 2;
```

```
a = 240  
a =  
a =      b =  
a =      c =  
a =      d =  
a =      e =
```

```
a = 240  
a =  
a =      b =  
a =      c =  
a =      d =  
a =      e =
```

Pointer-Arithmetik

- Berechnen Sie die Werte der Variablen nach Ausführung jeder Zeile für unsere beiden Testplattformen!



```
1 int* a = 240;  
2 a = a + 2;  
3 int* b = a++;  
4 int* c = --a;  
5 int* d = &a[4];  
6 int* e = (a+d) / 2;
```

a = 240	
a = 244	
a =	b =
a =	c =
a =	d =
a =	e =

a = 240	
a = 248	
a =	b =
a =	c =
a =	d =
a =	e =

Pointer-Arithmetik

- Berechnen Sie die Werte der Variablen nach Ausführung jeder Zeile für unsere beiden Testplattformen!



```
1 int* a = 240;  
2 a = a + 2;  
3 int* b = a++;  
4 int* c = --a;  
5 int* d = &a[4];  
6 int* e = (a+d) / 2;
```

```
a = 240  
a = 244  
a = 246    b = 244  
a =         c =  
a =         d =  
a =         e =
```

```
a = 240  
a = 248  
a = 252    b = 248  
a =         c =  
a =         d =  
a =         e =
```

Pointer-Arithmetik

- Berechnen Sie die Werte der Variablen nach Ausführung jeder Zeile für unsere beiden Testplattformen!



```
1 int* a = 240;  
2 a = a + 2;  
3 int* b = a++;  
4 int* c = --a;  
5 int* d = &a[4];  
6 int* e = (a+d) / 2;
```

```
a = 240  
a = 244  
a = 246    b = 244  
a = 244    c = 244  
a =        d =  
a =        e =
```

```
a = 240  
a = 248  
a = 252    b = 248  
a = 248    c = 248  
a =        d =  
a =        e =
```

Pointer-Arithmetik

- Berechnen Sie die Werte der Variablen nach Ausführung jeder Zeile für unsere beiden Testplattformen!



```
1 int* a = 240;  
2 a = a + 2;  
3 int* b = a++;  
4 int* c = --a;  
5 int* d = &a[4];  
6 int* e = (a+d) / 2;
```

```
a = 240  
a = 244  
a = 246    b = 244  
a = 244    c = 244  
a = 244    d = 252  
a =         e =
```

```
a = 240  
a = 248  
a = 252    b = 248  
a = 248    c = 248  
a = 248    d = 264  
a =         e =
```

Pointer-Arithmetik

- Berechnen Sie die Werte der Variablen nach Ausführung jeder Zeile für unsere beiden Testplattformen!



```
1 int* a = 240;  
2 a = a + 2;  
3 int* b = a++;  
4 int* c = --a;  
5 int* d = &a[4];  
6 int* e = (a+d) / 2;
```

```
a = 240  
a = 244  
a = 246    b = 244  
a = 244    c = 244  
a = 244    d = 252  
a =         e =
```

```
a = 240  
a = 248  
a = 252    b = 248  
a = 248    c = 248  
a = 248    d = 264  
a =         e =
```

Diese Zeile erzeugt einen Compilerfehler, da der Operator + nicht mit zwei Operanden vom Typ int* verwendet werden kann

Pointer-Arithmetik

- Berechnen Sie die Werte der Variablen nach Ausführung jeder Zeile für unsere beiden Testplattformen!



Ohne den Referenzoperator kann unser Programm abbrechen, da wir auf Speicher zugreifen, den wir zuvor nicht reserviert haben

```
1 int* a = 240;  
2 a = a + 2;  
3 int* b = a++;  
4 int* c = --a;  
5 int* d = &a[4];  
6 int* e = (a+d) / 2;
```

244

a = 244	c = 244
a = 244	d = 252
a =	e =



a = 240	
a = 248	
a = 252	b = 248
a = 248	c = 248
a = 248	d = 264
a =	e =

EIN BEISPIELPROGRAMM

Beispielprogramm

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// Definiere eigenen "Datentyp" player
struct player {
    char *name;
    int number;
};

// Funktion zum Anlegen eines Spielers
struct player *getPlayer(char *name, int number) {
    // Allokieren Speicher auf dem Heap
    struct player *p = malloc(sizeof(struct player));
    (*p).name = malloc(strlen(name) + sizeof(char));
    // Kopiere Name und Nummer
    strcpy(p->name, name);
    p->number = number;
    return p;
}
```

strlen gibt die
Länge des Namens
zurück,
zusätzlicher char
für null-byte (\0)

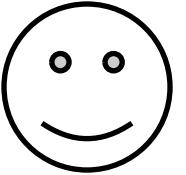
```
// Funktion zur Freigabe des allokierten Speichers
void deletePlayer(struct player *p) {
    free(p->name);
    free(p);
}

int main(void) {
    struct player *list[3];
    char name[256]; // Char array für das temporäre
    // Abspeichern des eingelesenen Namens
    for (int i = 0; i < 3; i++) {
        printf("Name von Spieler %d eingeben:\n", i + 1);
        fgets(name, 256, stdin); // Name einlesen
        list[i] = getPlayer(name, i);
        printf("Spieler %d: %s", i + 1, list[i]->name);
    }
    for (int i = 0; i < 3; i++) {
        // Allokieren Speicher auf dem Heap freigeben
        deletePlayer(list[i]);
    }
    return 0;
}
```

Funktionen zum Einlesen von Zeichenketten

➤ `char *fgets( char *restrict str, int count, FILE *restrict stream );`

- Liest Zeichen vom gegebenen Stream (z.B. stdin für Konsoleneingaben) und stoppt beim ersten Zeilenumbruch `\n`, am Ende des Datenstroms (EOF – End Of File), wenn ein Fehler auftritt oder wenn bereits `count-1` Zeichen eingelesen wurden
- Speichert die Zeichen (inkl. `\n`) und ein zusätzliches null-byte `\0` im gegebenen string und gibt einen Pointer auf das erste Zeichen zurück (falls kein Fehler auftritt)



➤ `char *gets( char *str );`

- Verhält sich wie `fgets` mit `stdin` als Datenstrom und „`count = ∞`“
 - Kann mehr Zeichen einlesen als im Buffer (z.B. `char name[256]` auf vorheriger Folie) reserviert
 - Dadurch können andere Speicherbereiche überschrieben werden! **Sicherheitsrisiko -> Never use gets!!**

➤ `int scanf( const char *restrict format, ... );`

- Liest von `stdin`, interpretiert die Zeichen je nach format (z.B. `%s` → „sequence of non-whitespace characters“) und speichert sie an die bei `...` angegebenen Stellen, z.B. `char *str`
- Wie schon bei `gets` kein bounds-checking → **Sicherheitsrisiko!**